SDR Platform — Experiment Operating Guide

Contents

SDR Platform — Experiment Operating Guide **1**
0. One-time setup (do once per host) . . . 1
1A. Application 1 — MARL / OSU random access . . . 2
1B. Application 1 — Federated Learning (MNIST) . . . 3
2. Application 2 — STC-AirComp (design only — not yet runnable) . . . 4
3. Application 3 — CLIP semantic communication . . . 4
4. Troubleshooting (the gotchas that actually bite) . . . 5
5. Quick-reference card . . . 5

SDR Platform — Experiment Operating Guide

A step-by-step runbook for operating **every application** on the shared SDR PHY. Each application has a **radio-free** path (validate the logic with no hardware) and a **hardware** path (two hosts + USRPs). Do the radio-free steps first — they catch 90% of mistakes before a radio is involved.

App	Folder	Built?	Radio-free	Hardware
1. Reliable link — **MARL / OSU**	`MARL_RA_Union/`, `python/`	yes	yes	yes (needs $\geq$3 USRP for real collision
1. Reliable link — **Federated Learning**	`python/fl.py`	yes	yes	yes
2. **STC-AirComp**	`STC_AirComp_Union/`	design	planned	planned
3. **CLIP semantic comm**	`CLIP_SemCom_Union/`	yes	yes	yes

Conventions in this guide: `$AP_IP` / `$SERVER_IP` / `$RX_IP` = the receiver/AP host's LAN IP; `serial=30CD424` and `serial=30CD3F7` = the two B210s; `addr=192.168.20.2` = the N210. **Always start the receiver/AP first**, then the transmitter.

---

0. One-time setup (do once per host)

0.1 Install dependencies

```
cd Hardware_update
./deploy/initialization.sh           # C++ PHY deps (UHD, Boost, FFTW3f, VOLK) + Python (numpy, matplotlib)
./deploy/initialization.sh --build   # also configure + compile build/sdr_system
```

0.2 Build the C++ PHY (`sdr_system`) — natively on each box

The binary is **architecture-specific** — a Mac (arm64/x86_64) binary will not run on the Linux lab host (`Exec format error`). Build on the machine that will run it:

```
cd Hardware_update/phy
conda deactivate 2>/dev/null || true      # Anaconda's cmake bakes a dead path and breaks `make`
rm -rf build && mkdir build && cd build
cmake ..            # if `cmake` is Anaconda's, use /usr/bin/cmake .. explicitly
make -j
ls ./sdr_system     # <- the binary
```

Rebuilding also regenerates `phy/python/sdr.py` and `PARAMETERS.md` (CMake post-build).

0.3 Build the block API `pyphy` (needed for the CLIP `pyphy` channel; optional otherwise)

```
cd Hardware_update/phy
bindings/build.sh                # DSP blocks only (builds anywhere)
WITH_UHD=1 bindings/build.sh     # + the Radio source/sink (on the lab host, needs UHD)
# On macOS run pyphy code as:  PYTHONPATH=phy/bindings arch -x86_64 python3 ...
```

0.4 Confirm the radios are visible

```
uhd_find_devices                            # lists connected USRPs
uhd_find_devices --args addr=192.168.20.2   # the N210 specifically
```

0.5 Pick a clean carrier (recommended before any RF run)

```
cd phy/python
python3 freq_scan.py --start 902 --stop 928 --step 1 --rx-args addr=192.168.20.2
# use the quietest highlighted frequency as your link freq (default is 915 MHz)
```

0.6 Golden rules (save yourself hours)

- **RX/AP first, TX second.** Keep both radios **warm** (started once, left running).
- **N210 rates are exact:** `--rx-rate 2e6 --symbol-rate 1e6` (1.6e6 does not snap on its 100 MHz clock).
- **Two-host ACK:** each side's `--ack-host` = the *peer* host's IP.
- **Free-running LOs:** single-carrier coherent QPSK ≈ 17% delivery. Use **DQPSK** (single-carrier) or **OFDM** (coherent QPSK, pilots track CFO). A shared 10 MHz reference makes coherent reliable.
- **Stuck RX won't Ctrl-C?** Press Ctrl-C again (escalating handler); emergency kill = `Ctrl-\` or `kill -9`.
- **`<frozen getpath>` crash** = stale shell CWD -> `cd` out and back in.

1A. Application 1 — MARL / OSU random access

RL agents share one channel; the **ACK is the reward** (delivered = success, timeout = collision/loss). Single-shot bursts (`--max-attempts 1`); the *policy* owns retransmission.

Step 0 — radio-free validation (no hardware)

```
cd applications/MARL_RA_Union
python3 ap_multi.py --self-test      # per-agent ACK routing (synthetic id stream)  -> PASS
python3 ap_multi.py --sim-test       # parser + end-to-end with a fake C++ sink       -> PASS
python3 slot_sync.py --self-test     # slot clock aligns 3 clients                    -> PASS

# offline decentralized run (one process per agent, mock medium):
python3 mock_medium.py --agents 2 --slots 600 &
python3 agent_node.py --mock --id 0 --slots 600
```

```
python3 agent_node.py --mock --id 1 --slots 600

# offline training (learns to transmit; prints P(transmit|queued) climbing):
python3 marl_train.py --mock --steps 400                       # single agent (A2C)
python3 marl_multi_train.py --mock --agents 4 --steps 800 \
        --coll-penalty 0.5 --compare-aloha "0.15,0.25,0.5"     # multi-agent vs ALOHA
```

Success: self/sim-tests PASS; in training `P(transmit|queued)` rises on a clean mock link; multi-agent collision rate drops and per-agent `P(transmit)` settles near `1/N`.

Step 1 — hardware, single agent (2 USRPs: 1 AP + 1 agent)

```
# AP host (warm receiver + ACK router):
python3 real_channel.py ap --rx-args serial=30CD3F7

# Agent host (trains online from real ACK/timeout):
python3 marl_train.py --tx-args serial=30CD424 --steps 150 --scheme DQPSK
```

Success: the agent fires real bursts; reward tracks real delivery; the model + reward plot save next to `--out`. (One agent has no collision partner — use Step 2 or the jammer for contention.)

Step 2 — hardware, decentralized multi-agent (3 USRPs: 1 AP + 2 agents)

```
# AP host — start BOTH the decoder/ACK router and the slot clock:
python3 ap_multi.py --agents 2 --scheme QPSK --rx-args serial=30CD3F7    # B210 AP
#   (N210 AP instead:  --rx-args addr=192.168.20.2 --rx-subdev A:0 --rx-rate 2e6 --symbol-rate 1e6 --scheme DQPSK)
python3 slot_sync.py --agents 2 --slot-ms 150                            # slot clock (port 5600)

# Each agent host (local-only obs; learned CSMA), pointing at the AP host:
python3 agent_node.py --id 0 --tx-args serial=30CD424 --scheme QPSK --ap-host $AP_IP --slot-host $AP_IP
python3 agent_node.py --id 1 --tx-args serial=<3RD>   --scheme QPSK --ap-host $AP_IP --slot-host $AP_IP
```

Success: AP prints `[BURST] id=… hex=…` per decoded frame and routes an ACK to that agent; $\geq$2 intents in a slot $\Rightarrow$ COLLISION (no ACK); agents learn to back off (collision rate falls).

Optional — add real contention with the jammer

```
python3 ../jammer/jammer.py --tx-args serial=<JAMMER> --freq 915e6 \
        --mode burst --interval 150 --duration 300 --tx-gain 80
```

1B. Application 1 — Federated Learning (MNIST)

Clients train locally and send **compressed model deltas** (float32 vectors) up to a server that FedAvg-aggregates and broadcasts the aggregate down.

Step 0 — radio-free

```
cd applications/FL_Union
python3 fl.py --mock --clients 2 --rounds 20                    # compressed (default) -> ~0.71->0.93
python3 fl.py --mock --clients 2 --rounds 20 --compress-ratio 0 # full model

# whole protocol over TCP (no radios), real server/client processes:
python3 fl.py server --clients 1 --rounds 20 --uplink tcp --downlink tcp &
```

```
python3 fl.py client --client-id 0 --clients 1 --rounds 20 --uplink tcp --downlink tcp \
        --net-host 127.0.0.1 --net-port 5700 --ack-host 127.0.0.1
```

Success: mock accuracy climbs ~0.71->0.93 in 20 rounds; TCP run reaches ~0.92 in 12 rounds.

Step 1 — hardware (server = N210 RX-only, client = B210 TX). Start the server first.

```
# Server host (N210): uplink over RF, model broadcast back over TCP
python3 fl.py server --clients 1 --rounds 20 --uplink wireless --downlink tcp \
        --rx-args addr=192.168.20.2 --rx-subdev A:0 --net-port 5700

# Client host (B210):  ack-host + net-host = the server's IP
python3 fl.py client --client-id 0 --clients 1 --rounds 20 --uplink wireless --downlink tcp \
        --tx-args serial=30CD424 --ack-host $SERVER_IP --net-host $SERVER_IP --net-port 5700
```

Success: each round the client's delta is delivered (ARQ), the server aggregates and broadcasts; accuracy rises round over round. Use `--waveform ofdm` for coherent QPSK if the SC link is marginal.

---

2. Application 2 — STC-AirComp (design only — not yet runnable)

This app is specified but **not implemented** (blocked on the paper's exact STLC precoding/combiner equations and on a synchronized 2-channel RX capture). See `STC_AirComp_Union/INTEGRATION.md`. When built, the operating sequence will be:

1. **DSP loopback (radio-free):** validate a 2-sensor -> 1-AP sum through a simulated 2-antenna AWGN channel (0 error at high SNR; NMSE-vs-SNR curve).
2. **`capture2` bring-up:** verify a synchronized 2-channel RX on a B210/X310 (2×2) captures one known TX.
3. **Single-sensor RF:** one B210 sensor -> 2×2 AP; STLC-encode, transmit, `capture2`, combine, recover the single value. Measure the timing/phase coherence budget.
4. **Two-sensor RF sum:** add a second B210, `slot_sync` them into one slot, recover `v1+v2`; report NMSE. Try **DBPSK** first (no shared clock), then BPSK + `--ref external` if coherence demands it.

To start building it, provide the paper's STLC matrices + estimator (see the INTEGRATION note's §7).

---

3. Application 3 — CLIP semantic communication

The base station encodes an image with **CLIP** into a float32 embedding and transmits **only the embedding**; the user classifies it (zero-shot) with no return link. Data-transfer archetype.

```
cd applications/CLIP_SemCom_Union
```

Step 0 — radio-free (no torch weights needed; uses the mock CLIP)

```
python3 semcom.py demo --mock                      # encode -> lossless channel -> classify (~0.99)

# reproduce the paper's accuracy-vs-noise (Fig. 3) through OUR real modem + FEC:
PYTHONPATH=../../phy/bindings arch -x86_64 python3 semcom.py demo --mock \
        --channel pyphy --scheme QPSK --fec turbo --snr-sweep 0,2,4,6,10

python3 semcom.py demo --mock --model-sweep        # 3-CLIP-model accuracy / payload / delay / energy
```

Success: clean accuracy ≈ 1.0; in the `pyphy` sweep accuracy climbs with SNR tracking BER (≈0 @0 dB -> ~0.7 @4 dB -> ~0.99 @6 dB); `int8` quant (`--quant int8`) is 4× smaller payload.

Step 1 — real CLIP weights (optional; torch already installed)

```
pip install -r requirements.txt        # open_clip_torch, pillow, ftfy, regex
python3 semcom.py demo --model vit-l14 # drops --mock automatically -> real ViT-L/14
```

Step 2 — over the radio (RX host = N210, TX host = B210). Start RX first.

```
# RX host (N210): receive embeddings and classify locally
python3 semcom.py rx --rx-args addr=192.168.20.2 --rx-subdev A:0 --num 8

# TX host (B210): encode images, send each embedding (ack-host = RX host IP)
python3 semcom.py tx --tx-args serial=30CD424 --ack-host $RX_IP
```

Success: the RX prints `predicted=<class> (truth=<class>)` per embedding. The radio path uses reliable ARQ (lossless delivery). For the on-air noise->accuracy curve, use the `--channel pyphy` sweep in Step 0 (radio-free), or the phase-2 single-shot mode in the INTEGRATION note.

---

4. Troubleshooting (the gotchas that actually bite)

Symptom	Cause -> Fix
`Exec format error` running `sdr_system`	Wrong-arch binary -> rebuild natively on that box (§0.
`<frozen getpath>` Python crash	Stale shell CWD -> `cd` out and back in.
Sink fails the **rate check** at startup	The receive-only sink still checks the full chain -> set `t`
`ACQ ERROR / Filtered vector too short` (works at small `k`, fails at large)	Long frame's energy dips below the detector -> raise T
Single-shot burst gets **no ACK** (MARL)	Cold LO / free-running CFO -> keep the AP **warm**, use
Coherent **QPSK ~17% delivery**	Free-running LO -> use **DQPSK** (SC) or **OFDM** (cohere
CLIP accuracy ~0 even at high SNR (uncoded)	Any bit error wrecks the 512-float payload -> add FEC
RX won't stop on Ctrl-C	Press Ctrl-C again (escalating handler); emergency = `c`
Two processes on one host clash	Give distinct ports: ARQ `--ack-port`, FL downlink `--net-`
pyphy `Python.h not found` / arch mismatch (macOS)	Use `phy/bindings/build.sh` (adds `-isystem`, `python3-config`

---

5. Quick-reference card

```
# ---- radio-free (no hardware) ----
(cd applications/MARL_RA_Union && python3 ap_multi.py --sim-test)      # MARL routing
(cd applications/MARL_RA_Union && python3 marl_multi_train.py --mock --agents 4 --steps 800 --coll-penalty 0.5)
(cd applications/FL_Union && python3 fl.py --mock --clients 2 --rounds 20)  # FL
cd applications/CLIP_SemCom_Union && python3 semcom.py demo --mock   # CLIP
PYTHONPATH=../../phy/bindings arch -x86_64 python3 semcom.py demo --mock --channel pyphy --fec turbo --snr-sweep 0,2,4,6,10

# ---- hardware (RX/AP host first, TX host second) ----
# MARL single:   real_channel.py ap --rx-args serial=30CD3F7   |   marl_train.py --tx-args serial=30CD424 --steps 150
# MARL multi:    ap_multi.py --agents 2 --rx-args serial=30CD3F7 + slot_sync.py --agents 2 | agent_node.py --id N --ap-host $AP_
# FL:            fl.py server --uplink wireless --downlink tcp --rx-args addr=192.168.20.2 --rx-subdev A:0 | fl.py client --tx-a
# CLIP:          semcom.py rx --rx-args addr=192.168.20.2 --rx-subdev A:0 | semcom.py tx --tx-args serial=30CD424 --ack-host $RX
```

For engine options: `PARAMETERS.md` (auto-generated, complete). For the full PHY reference: `../SYSTEM_REFERENCE.pdf`.
For per-application design: each folder's `README.md` + `INTEGRATION.md`, and `applications/APPLICATIONS_INTRO.pdf`.